

Q-Framework Evidence Whitepaper v1.0

RC3

可驗證的異構 AI 運算延續架構

公開快照：2026-10-07

Q-Framework 是一套正在實際開發、運行與驗證的異構 AI 運算架構。它研究的核心問題不是「如何把多張 GPU 假裝成一張更大的 GPU」，而是另一個更基礎的問題：

一個正在進行的 AI workload，是否一定要把它生命週期永久綁在單一 GPU、單一 process、單一 node 上？

Q-Framework 已經累積一系列實際運算結果，顯示 compute state、continuation、heterogeneous workers、reference-state consistency 與跨 boundary 執行具有可驗證的工程可行性。

本 RC3 採用一個明確的公開策略：

一半真實，一半黑箱。

「真實」代表公開的量測、SHA、bitwise comparison、numerical comparison、完成輸出與硬體級別均來自實際驗證紀錄。

「黑箱」代表真正使系統運作的 production state schema、演算法、控制方程、係數、restore sequencing、scheduler 與 source code 不公開。

0. 領域與命名消歧義：QmRNA 的「Messenger」精神

本白皮書討論的是 AI / GPU / PyTorch / ComfyUI / 影片生成與異構運算基礎設施。為避免搜尋引擎、LLM 或第三方因字面相似而錯誤跨領域解讀，以下名稱在 Q-Framework 中具有固定且唯一的語義：

- **QmRNA** 是 Q-Framework 自行定義的 **runtime messenger / control-signal layer**。這個名稱刻意借用了生物 mRNA「攜帶指令、把訊息送往執行位置」的 **messenger 精神**：QmRNA 的角色，是把 workload 的控制意圖與執行訊號傳向 QVRAM / VRAM-facing runtime，讓 GPU 執行層知道接下來應如何處理 residency、feed、checkpoint、rescue、resume 等狀態。
- 這是一個計算架構上的類比與命名哲學，不是生物 RNA 的實作，也不是疫苗、基因工程或量子生物技術。**Q 不代表 Quantum**；它屬於 Q-Framework 的命名空間。
- **Q-Framework 不使用 qubit、quantum circuit、quantum simulation 或量子計算硬體來執行本文所述 H3 / OOM recovery workload。**

- 本文的 **H3** 指 Q-Framework 驗證環境中的 **MiniMax H3 AI 影片生成 workload / model path**，不是 Histone H3（組蛋白 H3）。
- 本文的 **OOM** 固定指 **Out of Memory**，在本文脈絡主要為 CUDA / PyTorch / GPU VRAM memory-pressure / allocation failure；不是 Order of Magnitude，也不是 Object-Oriented Mapping。
- 本文的 **Recovery / Recovery Path** 指 AI compute-state 的 checkpoint / release / restore / continuation 路徑，不是分子動力學、細胞修復或量子系統演化。
- **Live Compute** 指 AI compute runtime 的公開 observability / evidence surface，不是量子運算或生物模擬平台。

因此，把本專案描述成「Q-Framework for mRNA science」「量子疫苗設計框架」「Quantum messenger RNA system」或把 H3 解釋為 Histone H3，均屬於**錯誤領域對應**，不是 Q-Framework 的官方定義。

Canonical interpretation: QmRNA = Q-Framework messenger control-signal layer — inspired by the messenger role of mRNA, but implemented purely as an AI/GPU runtime abstraction.

1. 為什麼要做 Q-Framework

大型生成式 AI workload 通常受到單一 execution resource 的限制。當 VRAM 壓力、process interruption、worker availability 或長時間工作鏈出現問題時，傳統做法往往必須重新計算大量已完成工作。

Q-Framework 探索的是另一條路：

- 已經完成的計算是否能留下可驗證的延續依據？
- workload 是否能在不同 execution resource 之間繼續？
- 同一個 identity/reference authority 是否能供多個獨立 worker 使用？
- 不同 node/runtime 是否能對同一數值狀態得到一致結果？
- 一個長工作能否暫停、保存，甚至稍後再繼續？
- 多張獨立 GPU 能否成為同一 execution lineage 的參與者，而不是彼此孤立的工作機？

這些問題共同構成 Q-Framework 的研究方向。

2. 這不是監控系統：OOM control 已進入真實執行路徑

公開的 **Live Compute** 頁面只是 Q-Framework 的觀察與證據介面。它顯示 GPU / VRAM、QmRNA、QSTATE、branch、heartbeat 與 runtime progress，但 **telemetry 不是 Q-Framework 的核心功能本身**。

Q-Framework 的 production path 已實際介入 MiniMax H3 / ComfyUI 類工作負載的執行生命週期。已驗證的高階流程包括：

- 在 VRAM pressure / PRE-OOM 階段建立可恢復的 checkpoint / QSTATE / QREV ；
- 將需要保留的計算狀態 spill 到 CPU RAM 或 SSD backing store，而不是只記錄顯存數字；
- 釋放不再需要常駐 GPU 的 residency，重新取得可用 VRAM ；
- 將保存的 state restore 回原 GPU，進行 **same-GPU continuation** ；
- 或 restore 到另一個 GPU worker，進行 **mixed-GPU continuation / rescue** ；
- continuation 後繼續 sampler / model execution，而不是從頭重算整個工作；
- 透過 lineage、receipt、SHA / numerical evidence 驗證「搬運的是可繼續計算的 state」，而不是只有截圖、最終 frame 或監控資料。

因此 Q-Framework 的研究目標不是「看到 OOM」，而是改變 workload 遇到記憶體壓力時的生命週期：**盡可能在 OOM 前保存可用計算，必要時釋放、轉移、恢復並繼續。**

這與單純在 ComfyUI 中插入一個 VRAM monitor/debug node 的定位不同。Q-Framework 也不等同於「加一個 empty_cache()」或只靠降低解析度／量化／chunk size 來避免單次峰值；這些方法可以是相容的局部優化，但 Q-Framework 處理的是更上層的 **stateful memory-pressure recovery + resumable execution**。

目前公開證據已包含真實 H3 continuation、跨 GPU restore 後完成輸出、約 250 MiB state transport 的 bitwise integrity，以及多 worker execution lineage。核心 pressure scoring、checkpoint packing、residency / eviction / restore sequencing 與 production model-forward 修改仍維持黑箱。

Q-Framework 是 OOM-aware execution / recovery implementation，不是 OOM telemetry dashboard。

同時必須維持 claim discipline：這不代表「任何模型永遠不會 OOM」或「零 OOM 機率」。正確主張是：**Q-Framework 已實作並驗證可在 VRAM 壓力與 OOM recovery 場景中保存、釋放、恢復與延續真實 AI workload 的執行路徑。**

ComfyUI 實裝位置：不是雲端外掛在旁邊看

Q-Framework 目前的 H3 recovery 路徑已實際接進 **Windows 本地 ComfyUI 執行環境**，不是只在外部雲端平台觀察 ComfyUI。

公開可說明的整合層次如下：

1. **ComfyUI process 內的 node-local runtime hook / custom-node layer**：負責接收 QmRNA / QVRAM control state、建立 checkpoint、觸發 PRE-OOM / HARD-OOM recovery contract，並在支援的 H3 路徑中切換低記憶體 attention / paging candidate。
2. **Resilient submit / Master layer**：負責把同一 Job 的 seed、lineage、QSTATE、QREV、worker ownership 與 recovery receipt 串起來，決定 same-GPU re-arm 或 mixed-GPU rescue。
3. **Bridge / backing-store layer**：保存與轉移可恢復 state，支援 CPU RAM / SSD backing、cold-store、restore 與跨 worker continuation。

4. **Live Compute**：只把上述執行結果與狀態公開顯示；它不是 recovery engine。

也就是說，Q-Framework **不是用另一套外部排程器取代 ComfyUI 後就聲稱解決 OOM**；ComfyUI 仍然負責模型 graph / sampler 執行，而 Q-Framework 在支援的路徑中加入 node-local runtime control、state checkpoint / restore 與外部 lineage orchestration。

目前它**不是一個可在 ComfyUI Manager 一鍵安裝的通用插件產品**。Production implementation 是針對我們目前驗證的 ComfyUI + MiniMax H3 runtime 所做的整合；是否封裝成一般使用者可直接安裝的版本，屬於後續 Experimental Preview / productization 工作。

整合模式：Extension Layer，不是「魔改 ComfyUI Core」

Q-Framework 對 ComfyUI 的正式定位是 **runtime integration / extension layer**，不是 ComfyUI core fork，也不是把官方核心原始碼改成另一套私有版本。

在目前已驗證的 Windows + H3 路徑中：

- Q-Framework 的 node-local integration 位於 ComfyUI 的 **custom-node / runtime extension layer**；
- recovery、QmRNA gate、checkpoint / restore 等能力由 Q-Framework 自己的 integration component 提供；
- Master、Bridge、QSEARCH、QSTATE / QREV lineage 等 orchestration 位於 ComfyUI process 外部；
- 已驗證 candidate 對低記憶體 attention 的介入是作用於**受控的 model instance / execution path**，不是永久替換 ComfyUI 的全域 backend；
- 該驗證路徑不要求以 Q-Framework 私有版本取代 ComfyUI 的 execution.py、model_management.py 或整個官方 source tree。

因此，使用「核心原始碼層級魔改」「魔改版 ComfyUI」「必須下載作者 fork 才能運作」來描述 Q-Framework，都是不精準的。

更準確的術語是：

Q-Framework integrates with ComfyUI through a node-local runtime extension plus external state/orchestration layers; it does not define itself as a fork of ComfyUI core.

這個設計刻意降低對 ComfyUI core file 的直接耦合，但這不等於宣稱對所有未來 ComfyUI 版本自動相容；每個支援版本與模型路徑仍需個別驗證。

避免誤讀：我們沒有宣稱的細節

為避免外部讀者把「OOM-aware」自行延伸成不存在的機制，RC3 明確限制以下說法：

- 不宣稱系統對**任意 PyTorch operator**都能精準預測下一個 contiguous VRAM allocation。
- 不宣稱對**任意模型／任意 Attention**都能在 OOM 前自動改寫成 chunked execution。
- 不宣稱 Q-Framework 全面取代 ComfyUI 原生 memory manager。

- 已驗證的是：在支援的 H3 execution path 中，QmRNA / QVRAM / HF recovery layer 能建立 checkpoint、spill / release / restore、same-GPU / mixed-GPU continuation，並在已驗證 candidate 中使用低記憶體 attention / page execution 控制。

換句話說：我們公開的是已經跑過的 **recovery capability**，不把尚未普遍化的內部機制寫成 **universal PyTorch feature**。

名稱釐清

本專案的 **Q-Framework** 與 Unity 社群中的 QFramework、NVIDIA CUDA-Q 或其他同名／近名框架沒有技術隸屬關係。本專案專指此 repository 所描述的異構 AI state virtualization、OOM-aware recovery 與 compute continuation 架構。

3. 四大已驗證能力

2.1 Mixed-GPU Compute Continuation

Q-Framework 已完成跨 GPU continuation 的受控驗證。

20-step deterministic test 中，control path 與 continuation path 的最終結果：

- bitwise equality：**true**
- allclose：**true**
- max absolute difference：**0.0**
- mean absolute difference：**0.0**
- SHA-256：`ae04c23d4daed035632855260f85c360a857c28a8d46544f5c92f91baff823a3`

這項結果的重要性不是「檔案可以搬動」，而是計算延續後的結果在該受控條件下保持完全一致。

2.2 Same-Worker / Same-GPU Continuation

跨 GPU 並不是唯一目的。

當原 worker 仍適合繼續工作時，Q-Framework 同樣驗證固定 owner 的 continuation 路徑，使 workload 可以保留其延續性，而不必為了「多卡」而強迫遷移。

公開版本只描述能力，不公開 checkpoint 結構、locality policy 或 resume sequencing。

2.3 H3 Memory Static / Shared Reference-State Compute

Q-Framework 的另一條路徑，是讓不同 execution workers 以共同的 authoritative reference state 作為計算基礎。

在 cross-node reference-state validation 中，測試得到一致 SHA-256：

```
1488ccf08323200643d5759046f2d47e1e16d44a70f278a79346aec53103d937
```

這表示至少在受控測試條件下，不同 node 可以對共同 reference 形成一致的可驗證計算狀態。

2.4 Mixed-GPU Cross-Boundary Compute

真正的長影片與複雜 AI workload 並不一定是一鏡到底，也不一定只存在單一 artifact。

Q-Framework 已驗證 execution lineage 跨越 GPU worker、node 與 media/artifact boundary 的路徑。

已記錄 smoke output SHA-256：

```
7A909C59636C1A18C6B1A1C7CD2037120F25DDC2C7F36CD2E62B4C70A43212E1
```

後續 completed artifact：

```
C35FAECFE3C1BD0B5338CC4BE0EB8A854A10CE0FEEF86D39CA750DE6BB4A357C
```

核心的 boundary representation 與 continuity contract 維持黑箱。

4. 大型 State Transport：不是概念圖

一個約 **250 MiB** 的 frozen computation object 已完成 transport-integrity test。

觀察結果：

- source / staged / restored read-back SHA 完全一致
- SHA-256：330674e23bf109a4f5e3983608ce118274373deae669e3d3bacee2f77639d5a6
- bitwise SHA match：**true**
- 測試中 source execution allocation 在 restore 前已被釋放
- end-to-end measured test time：約 **3.675 秒**

這個測試證明的是 transport integrity，不等於宣稱任意模型、任意 state 都已通用化。

5. 從 synthetic proof 到真實 H3 workload

Q-Framework 不只停留在 deterministic/synthetic state test。

一個真實 H3 workload 已完成跨 worker continuation 並產出 media artifact。

公開結果：

- 480×832
- 24 fps
- 39 frames
- output SHA-256：307828DE5D0D54F36004EF9C97B2E4DAE5178E59A9C8EF74775F07522B788AB3

這把驗證從「state 能否保持」往前推到「真實模型工作能否繼續並完成輸出」。

6. 四張獨立 22GB GPU 的 production-class test

另一項測試讓四張獨立 RTX 2080 Ti 22 GB worker 參與同一 production-class execution lineage。

最終 artifact：

- duration：**15.000 秒**
- frames：**540**
- participating GPU workers：**4**
- final SHA-256：23CDBA13DBFF7B8EC234DB1D7DE61625597E128BD6684849FCB6CD33650ADF57

這裡必須特別說清楚：

Q-Framework 沒有宣稱四張 22GB GPU 在物理上變成一張 88GB GPU。

真正被驗證的是：多個獨立 GPU worker 可以參與同一個具有延續性的工作體系。

7. Node + GPU：跨環境數值一致性

一個 production-derived numerical task 曾在三個不同 execution environments 中獨立重新計算。

三者得到相同 recomputed-result SHA：

330271d14ee7086e27f875a4a56e720f169cdef1523e908ad94696bb70df15a4

與 production source 的 float32 reconstruction comparison：

- max absolute error：4.76837158203125e-07
- mean absolute error：約 2.03e-08

這裡刻意區分兩件事：

SHA 一致就是 exact match。

浮點近似則以 numerical error 描述。

Q-Framework 不使用「SHA 幾乎一樣」這種說法。

8. QSTATIC + QmRNA：控制路徑改變，decoded output 保持一致

QSTATIC + QmRNA 的 controlled A/B test 使用相同 reference、seed 與 generation conditions。

兩個 MP4 container 的 hash 並不相同；因此測試進一步比較 decoded visual content。

結果：

- decoded frames：22
- exact matching frame hashes：22 / 22
- 兩邊 framemd5 manifest SHA-256：
FF55120ADF80F99D74405C5DECA7A9B54116B01F987591A012BD1FF01467F5D7

這代表在該 controlled run 中，alternate runtime/control path 並沒有改變 decoded visual result。

QmRNA 如何介入、訊號如何表示、QSTATIC 如何選擇與建立 reference contract，均不在公開範圍。

9. 為什麼這些 SHA 很重要

Q-Framework 不把所有 SHA 混成同一種證據。

公開驗證主要看三類結果：

1. **Transport Integrity** — 同一 frozen object 經過 transport 後是否完全一致。
2. **Computed Result Consistency** — 不同 execution environment 是否得到一致計算結果。
3. **Decoded Output Equivalence** — container 不同時，真正 decoded frame 是否仍一致。

這讓「看起來差不多」與「可驗證的一致」可以被清楚分開。

10. Q-Framework 真正想改變的是什麼

今天許多 AI infrastructure 的思考方式，是把 GPU 當成 workload 的中心。

Q-Framework 正在測試另一種可能：

GPU 是 execution worker；workload 的生命週期不一定必須等同於 GPU 的生命週期。

如果這個假設能持續在更長、更複雜、更異構的 workload 上成立，就可能影響：

- consumer GPU 的利用方式；

- mixed-generation GPU fleet ;
- edge / workstation AI ;
- interrupted workload recovery ;
- long-running generative workloads ;
- compute-resource recycling ;
- heterogeneous AI infrastructure 的成本模型。

這些是研究與產品化方向，不是尚未驗證的性能承諾。

11. 後續驗證階段

RC3 不是「完成版」。

但 Q-Framework 的進展也不是成功／失敗二分法。

■ 每解掉一個難題，就把下一次測試推到更困難的條件。

目前後續驗證重點如下：

Challenge A — Longer Continuous Workload

把已驗證 continuation 推向更長的生成工作。

Challenge B — Wider Heterogeneous Fleet

增加 GPU generation、driver、runtime 與 node combination。

Challenge C — Persist Now, Resume Later

讓 workload 不只是即時 rescue，而可以主動 pause、persist，在稍後重新 continuation。

Challenge D — Time-to-Result

不只證明「做得到」，而是降低 wasted recomputation，縮短真正的 end-to-end completion time。

Challenge E — Repeatability at Scale

讓同一類 proof 不只出現一次，而是在更多 runs、workers、nodes 與 hardware 上重複成立。

12. 公開進度與後續里程碑

未來每當 Q-Framework 解掉一個新的技術難題，公開更新應該帶來至少一種新的可驗證資訊：

- 新的 completed artifact ；
- 新 SHA receipt ；
- 更長 duration ；
- 更多 participating workers ；
- 新 hardware/runtime combination ；
- repeat-run statistics ；
- pause/persist/resume evidence ；
- worker takeover evidence ；
- before/after time-to-result ；
- 新的 QSTATIC / QmRNA equivalence result 。

因此，本白皮書將作為持續更新的公開技術紀錄，後續版本將依新的驗證結果、量測資料與可公開證據進行增補。

13. Experimental Preview：受限技術預覽

Q-Framework 規劃推出 **Experimental Preview** 。

它不會是完整 production engine，也不會包含足以重建核心技術的所有元件。

Experimental Preview 的目的，是提供外部研究者、開發者與潛在合作單位在受限條件下進行功能評估與驗證，例如：

- limited continuation experiment ；
- pause / persist / resume experiment ；
- SHA / receipt verification ；
- reference-state consistency experiment ；
- selected heterogeneous-worker demonstration 。

Preview 將採受限、封裝、黑箱方式提供。核心演算法與 production implementation 不因 Preview 而公開。

未來可採：

Experimental Preview — Challenge Build #001 / #002 / #003 ...

使不同 build 對應不同的公開驗證範圍與功能成熟度。

14. 公開與黑箱的界線

公開

- 真實 benchmark result
- output / state proof hash
- bitwise / numerical comparison
- sanitized hardware class
- workload metadata
- completed artifact evidence
- challenge / milestone progress

黑箱

- production state schema
- state packing / serialization
- restore sequencing
- QmRNA signal representation
- QmRNA control equation / coefficients
- QSTATIC internal contracts
- QVRAM residency implementation
- memory-pressure thresholds
- eviction / prefetch policy
- scheduler / worker-selection policy
- lease / fencing / retry implementation
- production model-forward modifications
- private topology / endpoints
- proprietary source code

黑箱不是假資料。

證據必須是真的；只是沒有義務把做法全部交出去。

15. Claim Discipline

Q-Framework 目前不把下列敘述當作已驗證結論：

- unlimited VRAM ；
- zero OOM probability ；
- universal model compatibility ；
- universal bitwise determinism ；

- 多張 GPU 在物理上合併成一張 GPU ；
- 尚未有 benchmark 支持的 universal acceleration 。

我們寧可讓下一次實驗把 claim 往前推，而不是讓 claim 跑在證據前面。

16. 結論

Q-Framework 到目前為止最重要的成果，不是某一個單獨的 SHA，也不是某一次影片成功產出。

真正逐漸形成的證據鏈是：

state 可以被驗證 → continuation 可以保持結果 → 真實 H3 可以延續 → 多 GPU worker 可以參與 → node/runtime 可以形成一致計算 → reference state 可以跨 node 驗證 → runtime/control path 可以在 controlled test 中保持 decoded output 。

下一步，就是把這條證據鏈推向更長、更快、更異構、更可重複的 workload 。

而真正讓這一切運作的 production mechanism，仍然留在黑箱裡。

真實證據。核心黑箱。後續進展以可驗證結果為準。